ALMUERZO A TIEMPO: SISTEMA WEB DE PEDIDOS ANTICIPADOS BASADO EN MICROSERVICIOS PARA REDUCIR LA CONGESTIÓN DE LA CAFETERÍA DE LA UNIVERSIDAD DE CUNDINAMARCA, SECCIONAL UBATÉ

Universidad de Cundinamarca, seccional Ubaté · Programa de Ingeniería de Sistemas y Computación · Sistemas Operativos

"Lunch on Time": a microservice-based web system for advance ordering to reduce lunchtime congestion at the Universidad de Cundinamarca cafeteria, Ubaté campus

Michell Daniela Caicedo
e-mail: [correo institucional]
Julian Esteban Ordoñez
e-mail: [correo institucional]
Yenderson Julian Silva Gomez
e-mail: [correo institucional]
Docente: Erick Sang Ramírez · Noviembre de 2026

RESUMEN: *En la cafetería de la Universidad de Cundinamarca, seccional Ubaté, la mayoría de estudiantes sale a almorzar a la misma hora y la atención se congestiona. Una encuesta a 14 estudiantes de Ingeniería de Sistemas y Computación mostró que el 93 % ha llegado tarde a clase por esperar en la cafetería, el 79 % se queda sin almorzar al menos a veces y el 86 % solo conoce el menú al llegar. Este proyecto propone "Almuerzo a Tiempo", un sistema web en el que el estudiante consulta el menú del día, hace su pedido, elige una franja de recogida de 15 minutos con cupo limitado y paga al recoger. El sistema se diseñó con dos microservicios en Python y FastAPI (menú y pedidos), que se comunican por API REST, se empaquetan con Dockerfile, se prueban con Docker Compose, se despliegan con réplicas en Docker Swarm sobre PostgreSQL y se versionan con Git. Se especificaron 15 historias de usuario con 46 criterios de aceptación, organizadas en 5 sprints. Se presentan el modelo de información, la arquitectura, la viabilidad técnica y económica, la matriz de riesgos y el plan de desarrollo y pruebas de un prototipo funcional, que se construye con Google Antigravity. Se espera que el sistema reduzca la espera en la hora pico y la ausencia a clases de la tarde, con un costo en efectivo menor a $45.000 si se aloja en un servidor de la universidad.*

PALABRAS CLAVE: *cafetería universitaria, Docker Swarm, microservicios, pedidos anticipados.*

ABSTRACT: *At the Universidad de Cundinamarca cafeteria (Ubaté campus), most students go to lunch at the same time and service becomes congested. A survey of 14 Systems Engineering students showed that 93 % have arrived late to class because of the cafeteria queue and 79 % skip lunch at least sometimes. This project proposes a web system where students check the daily menu, place an order, choose a 15-minute pickup slot with limited capacity and pay at pickup. It is built as two Python/FastAPI microservices (menu and orders) that communicate through a REST API, packaged with Dockerfiles, deployed with replicas on Docker Swarm over PostgreSQL and versioned with Git. The paper presents 15 user stories with 46 acceptance criteria, the information model, the architecture, a technical and economic feasibility study, a risk matrix and the development and test plan for a functional prototype built with Google Antigravity.*

KEYWORDS: *advance ordering, Docker Swarm, microservices, university cafeteria.*

1 INTRODUCCIÓN

La hora del almuerzo es el momento de mayor demanda en la cafetería de la Universidad de Cundinamarca (UdeC), seccional Ubaté. Como gran parte de las clases termina a la misma hora, los estudiantes llegan juntos, hacen una sola fila y tienen poco tiempo antes de la siguiente clase. El resultado es que muchos llegan tarde, comen a las carreras o se quedan sin almorzar [1].

El problema es importante porque afecta el bienestar y el desempeño académico: en la encuesta aplicada por el equipo, 13 de 14 estudiantes han llegado tarde o se han perdido parte de una clase por esperar en la cafetería. Estudios en otras universidades reportan el mismo patrón: tiempos de atención mayores al receso de los estudiantes por una sola caja y por recesos no escalonados [2], y filas largas con falta de mesas como problema común [3].

Las aplicaciones de pedidos anticipados se han propuesto como solución en cafeterías universitarias [4], [5]. Sin embargo, la literatura muestra que, si los pedidos superan la capacidad de preparación, la espera empeora [6], [7]. Por eso este proyecto plantea un sistema de pedidos anticipados con **cupo limitado por franja horaria**, basado en la teoría de colas [12] y construido con una arquitectura de microservicios en contenedores orquestados con Docker Swarm, que es el enfoque trabajado en la asignatura de Sistemas Operativos [8], [10], [20].

El artículo presenta el planteamiento del problema, el modelo de información, el estudio de viabilidad, la metodología, el estado del arte, el marco teórico, el diseño de la solución y los resultados obtenidos hasta ahora: el diagnóstico y el diseño completo, junto con el plan para construir y probar el prototipo.

2 PLANTEAMIENTO DEL PROYECTO

2.1 DESCRIPCIÓN DEL PROBLEMA

Entre las 11:30 a. m. y las 2:00 p. m. la cafetería de la UdeC Ubaté atiende a la mayoría de los estudiantes en pocos minutos. El 64 % de los encuestados ha vivido filas muy largas, el 43 % señala que todos llegan a la misma hora porque las clases terminan igual y el 36 % que hay poco personal atendiendo (Fig. 1). Aunque la respuesta más frecuente es esperar entre 5 y 10 minutos, el 36 % tiene menos de media hora libre para almorzar; para ellos, cualquier demora significa llegar tarde o no comer.

El problema central, acordado por el equipo en el diagnóstico [1], es la **congestión de la cafetería a la hora del almuerzo, tanto en la atención como en el espacio para comer** (Fig. 2). Sus causas son: la salida simultánea a almorzar por horarios no escalonados; una atención lenta en el pedido, el cobro y la entrega con una sola fila; el espacio y las mesas insuficientes; y el desconocimiento del menú y los precios hasta llegar (86 %). Sus consecuencias presentes son la impuntualidad en las clases de la tarde (93 %), la omisión del almuerzo o su reemplazo por paquetes (79 %), comer de pie o fuera de la cafetería y el estrés de estudiantes y personal. Si no se atiende, la consecuencia futura es una afectación sostenida del rendimiento académico y de la salud de los estudiantes, que aumenta a medida que crece la matrícula de la seccional.

El estudio relaciona dos variables principales: la **demora en recibir el almuerzo** (minutos de espera) y la **puntualidad en la clase siguiente** (porcentaje de estudiantes que llega tarde). La satisfacción general con el servicio es aceptable (3,8 sobre 5), lo que indica que el problema no es la calidad de la comida sino el tiempo. El pago no es una causa: el 93 % sabe que la cafetería acepta transferencias.

La pregunta de investigación es: **¿en qué medida un sistema web de pedidos anticipados con cupo por franja horaria reduce el tiempo de espera en la hora del almuerzo y la impuntualidad de los estudiantes de la UdeC Ubaté?** El proyecto delimita su alcance a la atención; la falta de espacio y el escalonamiento de horarios dependen de la administración y se dejan como recomendación.

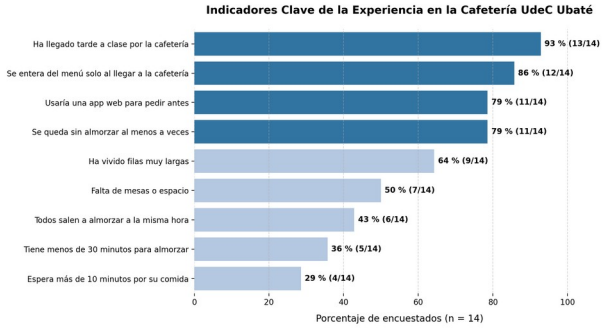


Fig. 1. Resultados principales de la encuesta del 6 de noviembre de 2026 (n = 14). Elaboración propia en Google Colab.

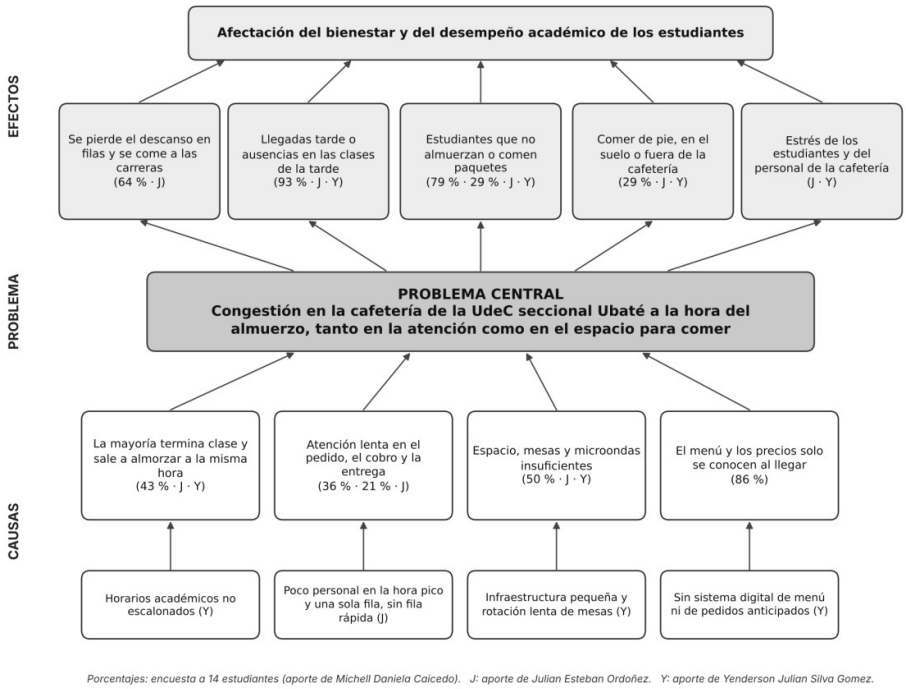


Fig. 2. Árbol de problemas del servicio de cafetería en la hora del almuerzo. Tomado del diagnóstico binario del equipo [1].

2.2 JUSTIFICACIÓN

El proyecto es **necesario** porque el 79 % de los estudiantes encuestados se queda sin almorzar al menos a veces y el 93 % ha llegado tarde a clase por la cafetería. Es **deseable** porque el 79 % afirma que usaría una aplicación web para pedir con anticipación y solo uno la rechazaría. Es **viable** porque se construye con herramientas libres, con los conocimientos de Docker y Linux vistos en la asignatura y con un costo en efectivo muy bajo (sección 4).

Entre las alternativas posibles, el sistema de pedidos anticipados con cupo por franja es la más adecuada dentro del alcance del equipo: publicar el menú por WhatsApp solo resuelve el desconocimiento del menú; contratar más personal o ampliar el espacio depende de la administración; y una aplicación de pedidos sin cupo puede empeorar la espera, como ocurrió en la Universidad de California en Los Ángeles [7]. Además, el proyecto permite aplicar en un caso real los contenidos de la asignatura: procesos concurrentes, contenedores, redes entre servicios y control de versiones.

2.3 OBJETIVOS

Objetivo general

Desarrollar un sistema web de pedidos anticipados con cupo por franja horaria, basado en dos microservicios desplegados con Docker, que reduzca la congestión de la cafetería de la UdeC seccional Ubaté a la hora del almuerzo.

Objetivos específicos

- Diagnosticar las causas y efectos de la congestión de la cafetería mediante una encuesta a estudiantes y el árbol de problemas.
- Especificar los requisitos del sistema en historias de usuario con criterios de aceptación verificables.
- Diseñar la arquitectura de microservicios, el modelo de información y las interfaces del sistema.
- Implementar los microservicios de menú y de pedidos con sus APIs REST, imágenes Docker, orquestación con Docker Swarm y control de versiones en Git.
- Evaluar el prototipo con pruebas automáticas por criterio de aceptación y una prueba de aceptación con usuarios que simulan los roles de estudiante y encargado.

2.4 ALCANCE Y LIMITACIONES

El sistema cubre el ciclo completo de un pedido anticipado: el encargado publica el menú; el estudiante consulta, pide, elige franja y, si lo necesita, cancela; el encargado prepara, marca como listo y entrega registrando el medio de pago. **No incluye** pago en línea (el pago se hace al recoger, en efectivo o transferencia), inicio de sesión institucional, notificaciones por correo o WhatsApp, aplicación móvil nativa ni control de inventario de ingredientes. Tampoco resuelve la falta de mesas, que depende de la infraestructura. La fila tradicional se mantiene para quien no use el sistema. El resultado del proyecto es un **prototipo funcional** listo para una prueba piloto; su lanzamiento en la cafetería queda fuera del alcance de este proyecto de clase.

2.5 REQUISITOS

Los requisitos funcionales salen de las 15 historias de usuario (Anexo A), con 46 criterios de aceptación que sirven como casos de prueba. Seis historias corresponden al microservicio de menú (registrar producto, consultar y filtrar el menú, marcar agotado, cambiar precio y registrar unidades) y nueve al de pedidos (hacer pedido, número consecutivo, franja de recogida, pendientes, marcar listo, consultar estado, entrega y pago, cancelación y cupo por franja). La Tabla 1 resume los requisitos no funcionales.

TABLA 1
Requisitos no funcionales

ID	Requisito	Verificación
RNF-01	Página del estudiante usable en celular (desde 360 px); pedido en menos de 1 minuto	Prueba con 3 usuarios
RNF-02	Respuesta de cada endpoint en menos de 1 s con 50 pedidos por franja	Medición con script
RNF-03	Si menu-ms falla, no se registra el pedido y se informa al estudiante	Detener contenedor
RNF-04	Pedidos simultáneos sin números repetidos ni sobreventa	Prueba de concurrencia
RNF-05	Se levanta con docker compose up (desarrollo) y docker stack deploy (Swarm)	Equipo limpio
RNF-06	Una base de datos, un Dockerfile y pruebas por microservicio	Revisión del repositorio
RNF-07	Documentación de cada API en /docs (Swagger)	Abrir /docs
RNF-08	Datos personales mínimos (nombre y código), según la Ley 1581 de 2012	Revisión del modelo
RNF-09	Si una réplica falla, el servicio sigue respondiendo y Swarm la reemplaza	Detener una réplica
RNF-10	Publicar una versión nueva sin apagar el sistema (rolling update)	docker service update

3 MODELO DE INFORMACIÓN Y ESTRUCTURAS DE DATOS

3.1 MODELO CONCEPTUAL DE INFORMACIÓN

Entidades, atributos y relaciones. El modelo tiene cinco entidades repartidas entre los dos microservicios; cada uno es dueño de sus datos y ninguno lee la base del otro (Tabla 2).

TABLA 2
Entidades del modelo conceptual

Entidad	Atributos principales	Dueño
Producto (products)	id, name, category (lunch, drink, snack), price, status (AVAILABLE, SOLD_OUT), stock	menu-ms
Pedido (orders)	id, date, number, customer_name, student_code, slot, status (RECEIVED, READY, DELIVERED, CANCELLED), payment_method, total, created_at	orders-ms
Ítem (order_items)	order_id, product_id, product_name y unit_price copiados, quantity	orders-ms
Franja (slot)	start time (HH:MM), 15 min, pedidos ocupados	orders-ms (calculada)

Entidad	Atributos principales	Dueño
Configuración (config)	slot_capacity, start_time, end_time	orders-ms

Las relaciones son: un **Pedido** tiene uno o varios **Ítems** (1:N); cada **Ítem** se refiere a un **Producto** (N:1), pero esa relación cruza microservicios, por lo que se guarda solo el id y una copia del nombre y del precio; cada **Pedido** pertenece a una **Franja** (N:1), y cada **Franja** admite como máximo el **cupo** de la **Configuración**. La decisión de persistencia se registra en el Anexo B (ADR-02).

Reglas de negocio.

- RN-01: el precio es mayor a 0 y el nombre del producto es único sin distinguir mayúsculas.
- RN-02: un producto Agotado o sin unidades no se puede pedir; al llegar a 0 unidades pasa a Agotado y al cancelar un pedido se devuelven sus unidades.
- RN-03: el número de pedido es consecutivo por día, empieza en 001 y nunca se repite.
- RN-04: solo se pueden elegir franjas que empiecen dentro de 15 minutos o más y que no estén llenas.
- RN-05: un pedido pasa por Recibido, Listo y Entregado; solo se cancela en Recibido.
- RN-06: un pedido conserva el precio con que se hizo aunque el producto cambie de precio.
- RN-07: para entregar se exige el medio de pago (efectivo o transferencia); para cancelar, el código del estudiante.

3.2 JUSTIFICACIÓN DE ESTRUCTURAS DE DATOS

Selección del núcleo. El núcleo del sistema es la **agenda de franjas**: decidir, en el momento de confirmar, si una franja tiene cupo, qué número le corresponde al pedido y si hay unidades de cada producto. Se evaluaron dos estructuras:

- **Cola de prioridad en memoria** (montículo ordenado por franja y número): inserción O(log n), pero se pierde al reiniciar el contenedor y no se comparte entre las réplicas de Swarm.
- **Tabla relacional con índices B-tree** (PostgreSQL): índice único (date, number) e índice (date, status, slot). Inserción O(log n), número siguiente con MAX sobre el índice en O(log n), conteo de una franja en O(log n + k) y lista de pendientes ya ordenada en O(log n + m), sin ordenar en memoria.

Se eligió la tabla con índices porque persiste los datos en un volumen de Docker, la comparten todas las réplicas, ofrece transacciones atómicas y mantiene la misma complejidad logarítmica. Con 10 franjas de 15 minutos y un cupo de 10, el sistema maneja como máximo 100 pedidos al día, de modo que cada operación responde en milisegundos.

Exclusión mutua entre réplicas. Como orders-ms corre en 3 réplicas, un candado dentro del programa no sirve: cada réplica tendría el suyo. La sección crítica [13] se protege en la base de datos: al confirmar, la transacción toma un bloqueo consultivo por día (pg_advisory_xact_lock) antes de contar el cupo y calcular el número, y lo libera al terminar [21]. Las unidades se descuentan con una sola instrucción condicional (UPDATE products SET stock = stock - c WHERE id = x AND stock >= c), que es atómica y cuesta O(k log p) para un pedido con k productos entre p productos del menú.

4 ESTUDIO DE VIABILIDAD TÉCNICA Y ECONÓMICA

4.1 VIABILIDAD TÉCNICA

El equipo cuenta con lo necesario: tres portátiles con Windows o Linux, conocimientos de Python, el laboratorio de microservicios con Docker de la asignatura y herramientas de desarrollo asistido por IA (Google Antigravity). Todo el software es libre o gratuito: Python, FastAPI, PostgreSQL, nginx, Docker, Docker Swarm y Git. El clúster puede empezar con un solo nodo (docker swarm init) de 1 vCPU y 2 GB de RAM y crecer agregando nodos worker; para la demostración en clase se puede formar con los portátiles del equipo. En la cafetería solo se requiere un computador o tableta con navegador e internet en la caja.

Como el proyecto entrega un prototipo que aún no se lanzará, la viabilidad operativa (que la cafetería acepte atender una caja de pedidos anticipados en la hora pico) se validará con el personal antes de un eventual lanzamiento; por ahora es el principal riesgo a futuro (R1).

4.2 RECURSOS Y COSTOS ESTIMADOS

La Tabla 3 resume el presupuesto en pesos colombianos de 2026; el detalle y los supuestos están en el Anexo C. El trabajo del equipo es un aporte en especie, valorado a $15.000 por hora (supuesto del equipo, cerca de 1,6 veces el salario mínimo de 2026, $1.750.905 [16], por 182 horas al mes). El costo en efectivo depende de dónde se aloje el sistema.

TABLA 3
Resumen del presupuesto (COP, 2026)

Concepto	Valor
Trabajo del equipo (80 h)	$1.200.000
Uso de equipos e internet	$122.500
Total aporte en especie	**$1.322.500**
Efectivo, escenario A: servidor de la UdeC	$44.000
Efectivo, escenario B: servidor en la nube (1 año)	$677.600

4.3 RIESGOS, RESTRICCIONES Y ALTERNATIVAS

Se identificaron once riesgos (matriz completa en el Anexo C). Los de mayor exposición son: **R1**, que la cafetería no adopte el sistema (probabilidad media, impacto alto), que se mitigará validando el flujo con la encargada antes de un eventual lanzamiento; **R4**, que los pedidos saturen la preparación (media, alto), que se mitiga con el cupo por franja; **R7**, errores en el código generado con IA (media, alto), que se mitiga con una prueba automática por criterio de aceptación; y **R8**, que los seis días de desarrollo no alcancen (alta, alto), que se mitiga con sprints diarios, desarrollo asistido por IA y priorizando las historias de prioridad alta (sprints 1 a 3) como producto mínimo.

Restricciones: seis días de desarrollo (del 11 al 16 de noviembre de 2026), presupuesto en efectivo casi nulo, pago solo presencial y tratamiento de datos personales según la Ley 1581 de 2012 [19]. **Alternativas descartadas** (Anexo B): una aplicación monolítica, una aplicación móvil nativa, una pasarela de pagos en línea, publicar solo el menú por WhatsApp y una fila virtual sin pedido.

5 METODOLOGÍA

El proyecto es una **investigación aplicada** con enfoque mixto: un diagnóstico descriptivo cuantitativo y el desarrollo de una solución tecnológica con metodología ágil Scrum [11].

5.1 VARIABLES, PARTICIPANTES E INSTRUMENTOS

TABLA 4
Definición operacional de variables

Variable	Definición operacional	Medida
V1 Tiempo de espera	Minutos desde que el estudiante llega a la cafetería hasta recibir su comida	Encuesta (P5); cronometraje en el piloto
V2 Impuntualidad	% de estudiantes que ha llegado tarde a clase por la cafetería	Encuesta (P11)
V3 Omisión del almuerzo	% que se queda sin almorzar al menos a veces	Encuesta (P12)
V4 Conocimiento del menú	% que conoce el menú antes de llegar	Encuesta (P8)
V5 Intención de uso	% que usaría una app web de pedidos	Encuesta (P13)
V6 Tiempo para pedir	Segundos para completar un pedido en la página	Prueba de usabilidad

Participantes. 14 estudiantes del programa de Ingeniería de Sistemas y Computación de la UdeC Ubaté, seleccionados por muestreo no probabilístico por conveniencia; 10 compran en la cafetería al menos una vez por semana. Criterio de inclusión: ser estudiante activo de la seccional. Por su tamaño y origen, la muestra no es representativa de toda la seccional; es un diagnóstico exploratorio.

Recolección. Encuesta anónima de 14 preguntas en Google Forms, aplicada el 6 de noviembre de 2026 y difundida por WhatsApp [1]. Se complementó con los aportes de los tres integrantes como usuarios del servicio.

Análisis. Estadística descriptiva (frecuencias y porcentajes) con Python (pandas y matplotlib) en Google Colab. Las preguntas de selección múltiple se contaron por opción.

5.2 DISEÑO Y FASES

- **Fase 1, diagnóstico:** encuesta, árbol de problemas y estado del arte [1].
- **Fase 2, análisis:** 15 historias de usuario con criterios de aceptación y product backlog de 40 puntos.
- **Fase 3, diseño:** arquitectura, modelo de información, APIs, diagramas UML e interfaces.
- **Fase 4, desarrollo:** 5 sprints de un día, del 11 al 15 de noviembre de 2026 (Tabla 6), con código asistido por Google Antigravity y revisado por el equipo, y una rama de Git por historia.
- **Fase 5, pruebas:** una prueba automática por criterio de aceptación (pytest), pruebas de integración en Docker, pruebas de tolerancia a fallas en Swarm y pruebas manuales.
- **Fase 6, prueba de aceptación:** compañeros del grupo usan el prototipo como estudiantes y un integrante como encargado; se mide V6 y se recorre el flujo completo. La prueba piloto con la cafetería (V1) queda para un eventual lanzamiento.

5.3 ORGANIZACIÓN DEL EQUIPO

El equipo trabaja con Scrum. La Tabla 5 resume los roles; los tres integrantes desarrollan.

TABLA 5
Roles del equipo

Integrante	Rol	Responsabilidades
Michell Daniela Caicedo	Product Owner y líder técnica	Prioriza el backlog, toma las decisiones de diseño (ADR), desarrolla los microservicios, integra el código y aprueba los pull requests; despliegue en Swarm
Julian Esteban Ordoñez	Scrum Master y desarrollador	Administra el tablero de Trello, dirige el daily de 15 minutos, controla los tiempos de cada sprint; desarrolla la página web y la configuración de Docker
Yenderson Julian Silva Gomez	Responsable de calidad y desarrollador	Escribe y ejecuta las pruebas automáticas y manuales, dirige la prueba de aceptación y redacta el README y el manual de uso

TABLA 6
Plan de sprints (noviembre de 2026)

Sprint	Fecha	Historias	Pts.	Entregable
1	Mié 11	CF-MENU-0001, 0002, 0003	8	menu-ms en PostgreSQL, Dockerfile y Compose
2	Jue 12	CF-PEDI-0001, 0002; CF-MENU-0004	9	Pedidos con número diario
3	Vie 13	CF-PEDI-0003, 0004, 0005	8	Franjas y panel del encargado
4	Sáb 14	CF-PEDI-0006, 0007, 0008	7	Estado, entrega y cancelación
5	Dom 15	CF-MENU-0005, 0006; CF-PEDI-0009	8	Cupo, unidades, Swarm y prueba de aceptación
—	Lun 16			Entrega final y sustentación

Hipótesis de trabajo. Con el sistema, el estudiante que pide con anticipación recibe su almuerzo en menos de 2 minutos desde que llega a la caja de pedidos, frente a los 5 a 10 minutos que hoy es la respuesta más frecuente. Se verificará en una futura prueba piloto (V1).

Uso de inteligencia artificial. Por indicación del docente, el proyecto usa IA en todas las fases: Claude

para el análisis, el diseño y la redacción; Google Antigravity para generar el código y las pruebas; y el asistente de Google Colab para el análisis de datos. El equipo revisa cada resultado, ejecuta las pruebas y responde por el contenido.

6 ESTADO DEL ARTE

Congestión en cafeterías universitarias. García Martínez et al. [2] midieron 19 minutos entre el pedido y la entrega en una cafetería universitaria de México, más que el receso de los estudiantes, y lo atribuyeron a una sola caja, a cuellos de botella en la preparación y a recesos no escalonados; con mejoras de proceso redujeron los tiempos. La Universidad Nacional de Colombia, sede Medellín [3], señaló las filas largas y la falta de mesas como problema de sus cafeterías y abrió una convocatoria para hacerlas más eficientes.

Sistemas de pedidos en cafeterías universitarias colombianas y de la región. Torres-Cubillos y García-Macías [4] desarrollaron U-Coffee, una aplicación para la cafetería de la Universitaria Agustiniana en Bogotá, y Cuenca Chamba [5] construyó el backend de un sistema web y móvil de comandas para la cafetería de la Escuela Politécnica Nacional de Ecuador. Ambos confirman que la digitalización del pedido es factible en el contexto universitario, pero no limitan la cantidad de pedidos por franja.

Límites de los pedidos anticipados. Robbins [6] modeló el efecto de las aplicaciones de pedido anticipado en restaurantes de comida rápida y encontró que pueden aumentar la espera de quienes siguen en la fila tradicional. En la Universidad de California en Los Ángeles, el cambio a pedidos por celular generó esperas de hasta 90 minutos porque los pedidos superaron la capacidad de cocina [7].

Vacío identificado. Las soluciones revisadas digitalizan el pedido, pero no controlan la demanda en el tiempo. Este proyecto agrega el cupo máximo por franja de 15 minutos y el control de unidades, y lo implementa con microservicios replicados en Docker Swarm, lo que facilita desplegarlo en un servidor institucional y soportar la carga de la hora pico.

7 MARCO TEÓRICO

El proyecto se apoya en cuatro conceptos: la teoría de colas explica el problema, y la arquitectura de microservicios, los contenedores y el desarrollo ágil sustentan la solución.

Teoría de colas. La ley de Little establece que $L = \lambda \cdot W$: el número promedio de personas en el sistema (L) es igual a la tasa de llegada (λ) por el tiempo promedio en el sistema (W) [12]. Cuando todos los estudiantes llegan a la vez, λ se dispara y, con la misma capacidad de atención, la fila y la espera crecen. El cupo por franja reparte las llegadas en el tiempo y limita λ a un valor que la cafetería puede atender, y el pedido anticipado saca del momento pico el tiempo de pedir y elegir.

Arquitectura de microservicios. Consiste en construir una aplicación como un conjunto de servicios pequeños e independientes, cada uno con su propia responsabilidad y sus propios datos, que se comunican por la red [8]. La comunicación usa el estilo REST, que organiza la interacción en recursos identificados por URL y operaciones HTTP sin estado [9].

Contenedores. Docker empaqueta cada servicio con sus dependencias en una imagen que se ejecuta aislada gracias a mecanismos del kernel de Linux como los espacios de nombres (namespaces) y los grupos de control (cgroups) [10]. La imagen se describe en un Dockerfile, y Docker Compose define varios contenedores, su red interna y sus volúmenes en un solo archivo.

Orquestación. Docker Swarm agrupa varias máquinas (nodos manager y worker) en un clúster y ejecuta cada servicio como un conjunto de réplicas. Reparte las peticiones entre ellas con una red overlay y el routing mesh, reemplaza las réplicas que fallan y actualiza versiones de forma gradual (rolling update) [20]. Usa el mismo formato de archivo de Docker Compose, con una sección deploy.

Concurrencia. Cuando dos pedidos llegan al mismo tiempo compiten por recursos compartidos (el consecutivo, el cupo y las unidades). La sección crítica debe ejecutarse con exclusión mutua para evitar condiciones de carrera [13]. Con varias réplicas, el bloqueo debe hacerse en la base de datos compartida [21]. Además, cada microservicio es dueño de su base (patrón database per service) y, si una operación que toca los dos servicios falla a medias, se deshace con una acción de compensación, como en el patrón saga [22].

Desarrollo ágil. Scrum organiza el trabajo en sprints cortos con un product backlog priorizado [11], y las historias de usuario describen cada funcionalidad desde la necesidad del usuario, con criterios de aceptación que se pueden probar [17]. Git permite trabajar en ramas por historia y conservar el historial [14].

En síntesis, la teoría de colas justifica limitar la demanda por franja; los microservicios, REST y Docker dan una solución modular y portable; Swarm la hace tolerante a fallas y escalable; y Scrum con historias de usuario asegura que cada parte responda a una necesidad verificada.

8 DISEÑO DE LA SOLUCIÓN

El sistema tiene dos actores: el **estudiante**, que consulta, pide, consulta el estado y cancela sin iniciar sesión, y el **encargado de la cafetería**, que administra productos, prepara y entrega pedidos y define el cupo (Fig. 3).

8.1 ARQUITECTURA

El sistema se despliega como un stack de Docker Swarm (Fig. 4) con cinco servicios: **web** (nginx, 2 réplicas), que sirve las dos páginas y reenvía /api/menu y /api/orders; **menu-ms** (2 réplicas), con los productos; **orders-ms** (3 réplicas, porque recibe la carga de la hora pico), con pedidos, franjas, cupo y entregas; y una base **PostgreSQL** por microservicio, fijada al nodo manager con su volumen. Cada microservicio está hecho en Python 3.12 con FastAPI [15]. Swarm reparte las peticiones entre réplicas y, si una falla, la reemplaza.

La única comunicación entre servicios va de orders-ms a menu-ms, por REST, para reservar y liberar unidades; si menu-ms no responde en 3 segundos, el pedido no se registra. Si la reserva funciona pero guardar el pedido falla, orders-ms libera las unidades como acción de compensación [22].

8.2 APIS

Las APIs reciben y devuelven JSON y responden los errores con el código HTTP (400, 404, 409 o 503) y el mensaje exacto del criterio de aceptación. La Tabla 8 resume los endpoints; las figuras 3 a 7 se agrupan al final de esta sección.

8.3 COMPORTAMIENTO

Hacer un pedido es el único flujo en que participan ambos microservicios (Fig. 5): orders-ms valida datos, franja y cupo; pide a menu-ms reservar las unidades; si todo está disponible guarda el pedido con su número y responde al estudiante. Un pedido pasa por cuatro estados y solo se cancela mientras está Recibido (Fig. 6).

8.4 INTERFACES

La página del estudiante está pensada para el celular y la del encargado para el computador de la caja (Fig. 7). Los productos agotados y las franjas pasadas o llenas aparecen apagados y no se pueden elegir; la lista de pendientes se actualiza cada 10 segundos.

8.5 DESPLIEGUE Y CONTROL DE VERSIONES

Docker se usa en tres niveles. Cada microservicio tiene un **Dockerfile** que construye su imagen. Para desarrollar, **Docker Compose** levanta todo en un equipo con docker compose up --build. Para producción, las imágenes se publican en un registro (GitHub Container Registry) y el archivo docker-stack.yml se despliega en **Docker Swarm** con docker stack deploy -c docker-stack.yml lunch. La Tabla 7 resume la configuración de cada servicio en Swarm.

TABLA 7
Servicios del stack en Docker Swarm

Servicio	Imagen	Réplicas	Configuración
web	nginx:alpine	2	Puerto 8080 en el routing mesh
menu-ms	propia (Dockerfile)	2	Healthcheck /health; rolling update
orders-ms	propia (Dockerfile)	3	Healthcheck /health; rolling update; se escala con docker service scale
menu-db	postgres:16	1	Volumen; solo en el manager; contraseña con docker secret
orders-db	postgres:16	1	Volumen; solo en el manager; contraseña con docker secret

Todos los servicios usan restart_policy on-failure y una red overlay propia; las actualizaciones se hacen réplica por réplica, iniciando la nueva antes de detener la vieja, para no cortar el servicio. El repositorio en GitHub tiene una carpeta por microservicio (código, Dockerfile, requirements.txt y pruebas), la carpeta web, docker-compose.yml y docker-stack.yml. Se trabaja con una rama por historia (feature/CF-MENU-0001-register-product), revisión por otro integrante antes de unir a main y una etiqueta por sprint.

Convención de idioma y commits. Siguiendo las buenas prácticas de la industria, todo lo técnico se escribe en inglés: el repositorio (lunch-on-time), el código, las variables, las funciones, los archivos, los microservicios, los endpoints, las tablas, los campos y las ramas. Los commits siguen el formato **Conventional Commits** (por ejemplo, feat(menu): register product [CF-MENU-0001] o fix(orders): reject empty order). Los textos que ve el usuario, como botones y mensajes de error, quedan en español, porque el sistema lo usan estudiantes de Ubaté (ADR-08).

TABLA 8
Endpoints de los microservicios

Servicio	Método y ruta	Función	Historias
menu-ms	POST /products	Registrar producto	CF-MENU-0001
menu-ms	GET /menu?category=	Menú del día, completo o filtrado	CF-MENU-0002, 0006
menu-ms	PATCH /products/{id}/status	Agotado o Disponible	CF-MENU-0003
menu-ms	PATCH /products/{id}/price	Cambiar precio	CF-MENU-0004
menu-ms	PUT /products/{id}/stock	Unidades del día	CF-MENU-0005
menu-ms	POST /products/reserve y /release	Uso interno de orders-ms: verificar, descontar y devolver unidades	CF-PEDI-0001, CF-MENU-0005
orders-ms	GET /slots	Franjas del día: disponible, pasada o llena	CF-PEDI-0003, 0009
orders-ms	POST /orders	Crear pedido y asignar número	CF-PEDI-0001, 0002, 0003
orders-ms	GET /orders?status=	Pendientes o para entregar, por franja	CF-PEDI-0004
orders-ms	GET /orders/{number}	Estado de un pedido	CF-PEDI-0006
orders-ms	POST /orders/{number}/ready | /deliver | /cancel	Cambios de estado	CF-PEDI-0005, 0007, 0008
orders-ms	GET y PUT /config	Cupo y horario de franjas	CF-PEDI-0009

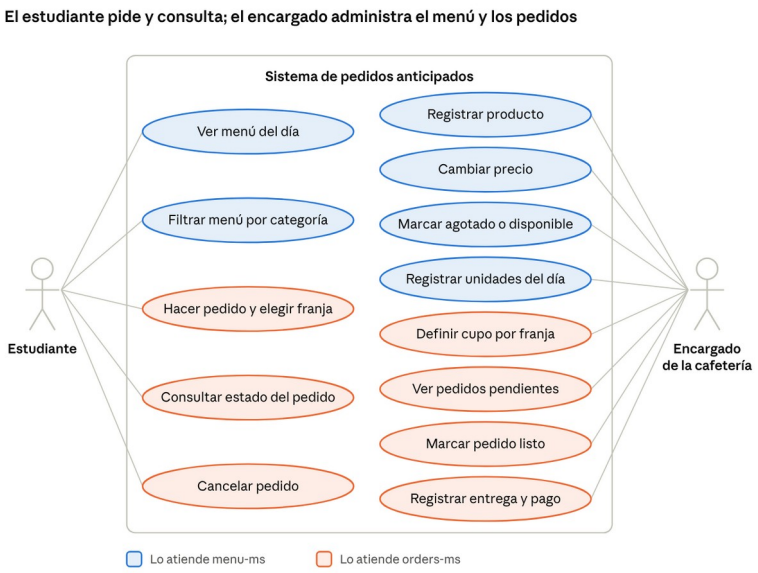


Fig. 3. Diagrama de casos de uso. Azul: atendido por menu-ms; naranja: atendido por orders-ms.

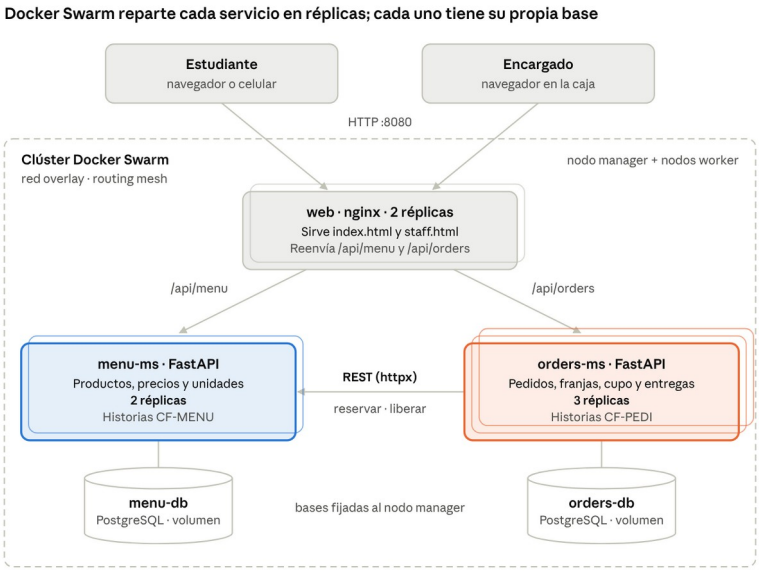


Fig. 4. Arquitectura del sistema desplegada en Docker Swarm.

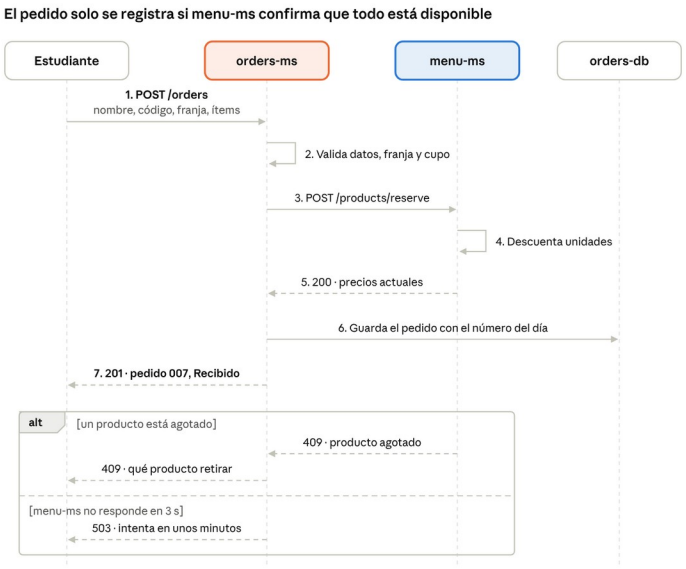


Fig. 5. Diagrama de secuencia: hacer un pedido.



Fig. 6. Diagrama de estados del pedido.

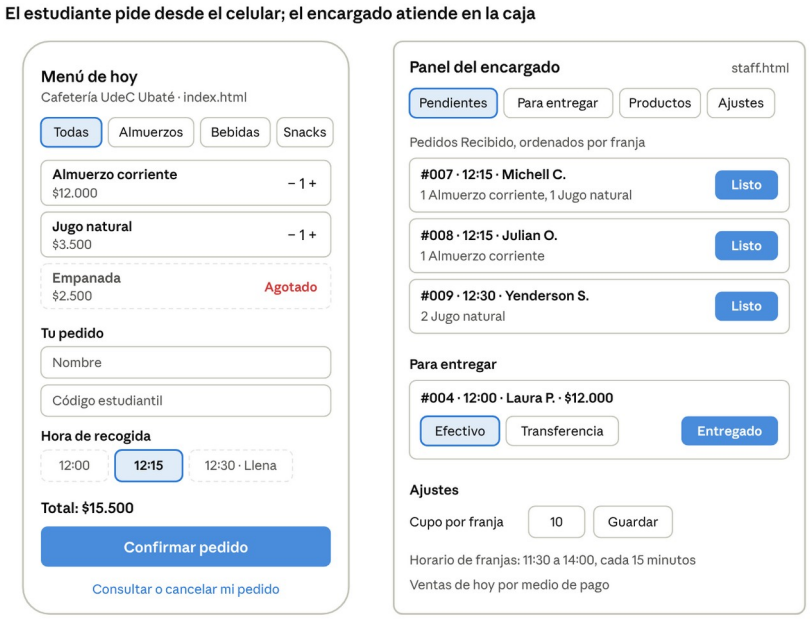


Fig. 7. Bocetos de las pantallas del estudiante y del encargado (nombres y precios de ejemplo).

9 RESULTADOS

Diagnóstico. La encuesta confirmó que el problema es el tiempo y no la calidad: la satisfacción promedio es 3,8 sobre 5, pero el 93 % ha llegado tarde a clase y el 79 % se queda sin almorzar al menos a veces (Fig. 1). También descartó dos supuestos iniciales del equipo: nadie reportó que se acabaran los productos y casi todos saben que se acepta transferencia. Por eso el pago en línea quedó fuera del alcance y el control de unidades pasó a prioridad media.

Requisitos y diseño. Se especificaron 15 historias de usuario con 46 criterios de aceptación y un backlog de 40 puntos en 5 sprints (Anexos A y C). Se diseñaron la arquitectura en Docker Swarm (tres servicios replicados y dos bases PostgreSQL), el modelo de información con cinco entidades y siete reglas de negocio, 20 endpoints REST, los diagramas de casos de uso, secuencia y estados, y las interfaces (Figs. 3 a 7). Las decisiones de diseño se registraron en ocho ADR (Anexo B).

Desarrollo. La construcción del prototipo empieza el 11 de noviembre con Google Antigravity, siguiendo este diseño y el plan de sprints (Tabla 6). Cada historia se cierra solo cuando pasan las pruebas de sus criterios de aceptación; los resultados, incluida la prueba de concurrencia del algoritmo central, se registran en el Anexo D.

TABLA 9
Indicadores para una futura prueba piloto

Indicador	Línea base	Meta
Espera desde que llega hasta recibir (pedido anticipado)	5 a 10 min (respuesta más frecuente)	Menos de 2 min
Estudiantes que llegan tarde a clase por la cafetería	93 %	Menos de 50 %
Estudiantes que conocen el menú antes de llegar	0 %	Más de 70 % de usuarios
Tiempo para hacer un pedido en la página	Sin dato	Menos de 60 s
Criterios de aceptación con prueba aprobada	0 de 46 (inicio)	46 de 46
Peticiones atendidas al detener una réplica	Sin dato	100 %

Las metas son supuestos del equipo y se ajustarán con el personal de la cafetería antes del piloto.

10 CONCLUSIONES

La congestión de la cafetería de la UdeC Ubaté en la hora del almuerzo se explica, sobre todo, por la llegada simultánea de los estudiantes y por una atención con una sola fila, y su efecto más grave es académico: casi todos los encuestados han llegado tarde a clase. Los datos permitieron pasar de suposiciones (pago lento, productos agotados) a un diagnóstico sustentado.

Un sistema de pedidos anticipados solo ayuda si controla la demanda en el tiempo. El cupo por franja de 15 minutos, sustentado en la ley de Little, es el aporte del proyecto frente a las soluciones revisadas.

La arquitectura de dos microservicios replicados en Docker Swarm es viable con los recursos del equipo, cuesta muy poco en efectivo y aplica de forma directa los temas de la asignatura: procesos concurrentes, exclusión mutua, contenedores, orquestación, redes entre servicios y control de versiones. Tener varias réplicas obligó a llevar la exclusión mutua a la base de datos, una lección central del diseño.

11 RECOMENDACIONES

- Antes de lanzar el sistema, validar el flujo y el cupo por franja con la encargada de la cafetería y hacer una prueba piloto de una semana.
- Ampliar la encuesta a otros programas, a docentes y al personal administrativo para confirmar los resultados.
- Proponer a la coordinación académica escalonar la hora de salida de algunas clases y a Bienestar Universitario ampliar el espacio para comer, ya que el software no resuelve esas causas.
- En una segunda versión, evaluar notificaciones por WhatsApp, inicio de sesión institucional y reportes de ventas para la cafetería.

AGRADECIMIENTOS

A los 14 estudiantes que respondieron la encuesta, al docente Erick Sang Ramírez por su orientación y al personal de la cafetería de la UdeC seccional Ubaté.

CONFLICTO DE INTERÉS

Los autores declaran que no tienen conflictos de interés.

REFERENCIAS

[1] M. D. Caicedo, J. E. Ordoñez y Y. J. Silva Gomez, "Diagnóstico binario: árbol de problemas del servicio de cafetería de la Universidad de Cundinamarca, seccional Ubaté," trabajo de clase, Sistemas Operativos, Univ. de Cundinamarca, Ubaté, 2026.

[2] R. García Martínez, M. N. Estrada Carbajal, B. Osorio Morales, M. A. García Jaramillo, A. Zarza Gómez y C. Primero Huerta, "Estrategia de reducción de tiempos y mejora en la logística en una cafetería universitaria," Ciencia Latina Rev. Cient. Multidisciplinar, vol. 8, no. 4, pp. 2301–2311, 2024, doi: 10.37811/cl_rcm.v8i4.12470.

[3] Universidad Nacional de Colombia, Sede Medellín, "Una cafetería más eficiente" [Convocatoria], Bienestar Universitario, 2025. [En línea]. Disponible: https://bienestaruniversitario.medellin.unal.edu.co/fomento socioeconomico/images/2025/9Septiembre/ beca_de_la_andi.pdf

[4] D. F. Torres-Cubillos y D. M. García-Macías, "Desarrollo de una aplicación, para el mejoramiento del servicio de la cafetería de la Universitaria Agustiniana - U-Coffee," trabajo de grado, Universitaria Agustiniana, Bogotá, 2020. [En línea]. Disponible: http://repositorio.uniagustiniana.edu.co/handle/123456789/ 1607

[5] M. E. Cuenca Chamba, "Desarrollo de sistema web y aplicación móvil para comandas en la cafetería EPN: desarrollo de un backend," trabajo de integración curricular, Escuela Politécnica Nacional, Quito, 2023. [En línea]. Disponible: http://bibdigital.epn.edu.ec/handle/15000/23829

[6] T. R. Robbins, "The impact of mobile order ahead apps on QSR queuing," J. Strategic Innovation and Sustainability,

vol. 16, no. 2, pp. 72–87, 2021, doi: 10.33423/jsis.v16i2.4298.

[7] E. Rusting, “Students air frustrations with UCLA Dining’s switch to mobile ordering,” Daily Bruin, 7 nov. 2023. [En línea]. Disponible: https://dailybruin.com/2023/11/07/students-air-frustrations-with-ucla-dinings-switch-to-mobile-ordering

[8] S. Newman, Building Microservices: Designing Fine-Grained Systems, 2.ª ed. Sebastopol, CA, EE. UU.: O’Reilly Media, 2021.

[9] R. T. Fielding, “Architectural styles and the design of network-based software architectures,” tesis doctoral, Univ. of California, Irvine, 2000.

[10] D. Merkel, “Docker: Lightweight Linux containers for consistent development and deployment,” Linux Journal, no. 239, 2014.

[11] K. Schwaber y J. Sutherland, La Guía de Scrum, 2020. [En línea]. Disponible: https://scrumguides.org

[12] J. D. C. Little, “A proof for the queuing formula: L = λW,” Operations Research, vol. 9, no. 3, pp. 383–387, 1961.

[13] A. Silberschatz, P. B. Galvin y G. Gagne, Operating System Concepts, 10.ª ed. Hoboken, NJ, EE. UU.: Wiley, 2018.

[14] S. Chacon y B. Straub, Pro Git, 2.ª ed. Apress, 2014. [En línea]. Disponible: https://git-scm.com/book/es/v2

[15] S. Ramírez, FastAPI (documentación oficial). [En línea]. Disponible: https://fastapi.tiangolo.com/es/

[16] Ministerio del Trabajo de Colombia, Decreto 1469 del 29 de diciembre de 2025, por el cual se fija el salario mínimo mensual legal para 2026.

[17] M. Cohn, User Stories Applied: For Agile Software Development. Boston, MA, EE. UU.: Addison-Wesley, 2004.

[18] DigitalOcean, “Droplet pricing,” 2026. [En línea]. Disponible: https://www.digitalocean.com/pricing/droplets

[19] Congreso de la República de Colombia, Ley Estatutaria 1581 de 2012, por la cual se dictan disposiciones generales para la protección de datos personales.

[20] Docker Inc., “Swarm mode overview,” Docker Docs. [En línea]. Disponible: https://docs.docker.com/engine/swarm/

[21] PostgreSQL Global Development Group, “Explicit locking,” PostgreSQL Documentation. [En línea]. Disponible: https://www.postgresql.org/docs/current/explicit-locking.html

[22] C. Richardson, Microservices Patterns. Shelter Island, NY, EE. UU.: Manning, 2018.

ANEXO A. Catálogo de necesidades e historias de usuario

Las historias se elaboraron sobre la plantilla institucional de historias de usuario (archivo Historias_Usuario_Cafeteria_UdeC.xlsx). La Tabla A1 relaciona cada necesidad detectada en el diagnóstico con las historias que la atienden; la Tabla A2 presenta las 15 historias y la Tabla A3 sus 46 criterios de aceptación.

TABLA A1
Catálogo de necesidades

ID	Necesidad	Evidencia	Historias
N1	Conocer el menú y los precios antes de llegar	86 % se entera al llegar	CF-MENU-0001, 0002, 0006
N2	Evitar la fila para pedir en la hora pico	64 % ha vivido filas muy largas	CF-PEDI-0001, 0002
N3	Recoger el almuerzo a una hora compatible con las clases	93 % ha llegado tarde; 36 % tiene menos de 30 min	CF-PEDI-0003, 0006
N4	No pedir productos que ya se acabaron	Supuesto inicial, no confirmado en la encuesta	CF-MENU-0003, 0005
N5	Que la cafetería no reciba más pedidos de los que puede preparar	43 % señala que todos llegan a la vez [6], [7]	CF-PEDI-0009, 0004, 0005
N6	Llevar el control de entregas y ventas del día	Necesidad del encargado (por validar)	CF-PEDI-0007
N7	Poder desistir del pedido sin generar desperdicio	Necesidad identificada por el equipo	CF-PEDI-0008
N8	Mostrar siempre el precio real	Necesidad del encargado (por validar)	CF-MENU-0004

TABLA A2
Historias de usuario

ID	Historia (rol, funcionalidad, razón)	Prioridad	Puntos	Sprint	Esc.
CF-MENU-0001	Como encargado de la cafetería, necesito registrar un producto con su nombre, categoría (almuerzo, bebida o snack) y precio, con la finalidad de que aparezca en el menú del día que consultan los estudiantes.	Alta	3	1	4
CF-MENU-0002	Como estudiante, necesito ver el menú del día con el nombre y el precio de cada producto, con la finalidad de saber qué hay y cuánto cuesta antes de ir a la cafetería.	Alta	3	1	3
CF-MENU-0003	Como encargado de la cafetería, necesito marcar un producto como agotado o volver a marcarlo como disponible, con la finalidad de que los estudiantes no pidan productos que ya no hay.	Alta	2	1	2
CF-PEDI-0001	Como estudiante, necesito hacer un pedido con uno o varios productos del menú, indicando la cantidad de cada uno, mi nombre y mi código estudiantil, con la finalidad de no tener que hacer la fila para pedir a la hora del almuerzo.	Alta	5	2	4
CF-PEDI-0002	Como estudiante, necesito recibir un número de pedido al confirmar mi pedido, con la finalidad de identificar mi pedido cuando vaya a recogerlo.	Alta	2	2	3
CF-MENU-0004	Como encargado de la cafetería, necesito cambiar el precio de un producto registrado, con la finalidad de que el menú muestre siempre el precio real.	Media	2	2	3
CF-PEDI-0003	Como estudiante, necesito elegir la hora de recogida de mi pedido entre franjas de 15 minutos, con la finalidad de recoger mi almuerzo cuando salga de clase, sin esperar.	Alta	3	3	3
CF-PEDI-0004	Como encargado de la cafetería, necesito ver la lista de pedidos en estado "Recibido" ordenados por hora de recogida, con la finalidad de preparar primero los pedidos que se recogen antes.	Alta	3	3	3
CF-PEDI-0005	Como encargado de la cafetería, necesito marcar un pedido como "Listo" cuando termine de prepararlo, con la finalidad de que el estudiante sepa que ya puede recogerlo.	Alta	2	3	2
CF-PEDI-0006	Como estudiante, necesito consultar el estado de mi pedido con su número, con la finalidad de saber si ya puedo recogerlo sin tener que hacer fila para preguntar.	Media	3	4	3
CF-PEDI-0007	Como encargado de la cafetería, necesito registrar que entregué un pedido y el medio con el que pagó el estudiante (efectivo o transferencia), con la finalidad de cerrar el pedido y llevar el control de las ventas del día.	Media	2	4	3
CF-PEDI-0008	Como estudiante, necesito cancelar mi pedido mientras esté en estado "Recibido", con la finalidad de que la cafetería no prepare comida que no voy a recoger.	Media	2	4	3
CF-MENU-0005	Como encargado de la cafetería, necesito registrar cuántas unidades hay disponibles de un producto en el día, con la finalidad de que el sistema deje de recibir pedidos de ese producto cuando se acabe.	Media	3	5	4
CF-PEDI-0009	Como encargado de la cafetería, necesito definir el número máximo de pedidos que se pueden recoger en cada franja de 15 minutos, con la finalidad de no recibir más pedidos de los que alcanzamos a preparar al mismo tiempo.	Media	3	5	3
CF-MENU-0006	Como estudiante, necesito filtrar el menú por categoría (almuerzos, bebidas o snacks), con la finalidad de encontrar rápido lo que quiero pedir.	Baja	2	5	3

TABLA A3

Criterios de aceptación (contexto, evento y resultado esperado)

Historia	Escenario	Contexto y evento	Resultado esperado
CF-MENU-0001	1. Registro exitoso	En caso que el encargado diligencie nombre, categoría y un precio mayor a 0. Cuando presione "Guardar".	El sistema guarda el producto en estado "Disponible" y lo muestra en el menú del día.
	2. Campos obligatorios vacíos	En caso que falte el nombre, la categoría o el precio. Cuando presione "Guardar".	El sistema no guarda el producto y muestra "Complete los campos obligatorios", señalando el campo que falta.
	3. Precio inválido	En caso que el precio sea 0, negativo o no sea un número. Cuando presione "Guardar".	El sistema no guarda el producto y muestra "El precio debe ser un número mayor a 0".
	4. Producto duplicado	En caso que ya exista un producto con el mismo nombre. Cuando presione "Guardar".	El sistema no crea el producto y muestra "Ya existe un producto con ese nombre".
CF-MENU-0002	1. Menú con productos	En caso que existan productos registrados para el día. Cuando el estudiante abra el menú.	El sistema muestra el nombre, la categoría y el precio en pesos de cada producto.
	2. Producto agotado	En caso que un producto esté marcado como "Agotado". Cuando el estudiante abra el menú.	El sistema muestra el producto con la etiqueta "Agotado" y no permite seleccionarlo.
	3. Menú sin publicar	En caso que no haya productos registrados para el día. Cuando el estudiante abra el menú.	El sistema muestra "La cafetería aún no ha publicado el menú de hoy".
CF-MENU-0003	1. Marcar como agotado	En caso que el producto esté "Disponible". Cuando el encargado presione "Marcar como agotado".	El sistema cambia el estado a "Agotado" y el producto deja de poder seleccionarse en el menú.
	2. Volver a disponible	En caso que el producto esté "Agotado". Cuando el encargado presione "Marcar como disponible".	El sistema cambia el estado a "Disponible" y el producto vuelve a poder seleccionarse.
CF-PEDI-0001	1. Pedido válido	En caso que todos los productos estén disponibles y el estudiante haya ingresado nombre y código. Cuando presione "Confirmar pedido".	El sistema registra el pedido en estado "Recibido" y muestra un resumen con productos, cantidades y valor total.
	2. Pedido vacío	En caso que el estudiante no haya agregado ningún producto. Cuando presione "Confirmar pedido".	El sistema no registra el pedido y muestra "Agrega al menos un producto".
	3. Producto agotado al confirmar	En caso que un producto del pedido se haya agotado después de agregarlo. Cuando presione "Confirmar pedido".	El sistema no registra el pedido e indica cuál producto se agotó para que lo retire.
	4. Menú no disponible	En caso que el microservicio Menú no responda. Cuando presione "Confirmar pedido".	El sistema no registra el pedido y muestra "No fue posible verificar el menú, intenta de nuevo en unos minutos".
CF-PEDI-0002	1. Asignación del número	En caso que el pedido se registre correctamente. Cuando el sistema confirme el pedido.	El sistema asigna un número consecutivo del día (por ejemplo, 001) y lo muestra en pantalla.
	2. Reinicio diario	En caso que sea el primer pedido de un nuevo día. Cuando el sistema confirme el pedido.	El sistema asigna el número 001.
	3. Pedidos simultáneos	En caso que dos estudiantes confirmen su pedido al mismo tiempo. Cuando el sistema registre ambos pedidos.	El sistema asigna un número diferente a cada uno.
CF-MENU-0004	1. Cambio exitoso	En caso que el nuevo precio sea un número mayor a 0. Cuando el encargado presione "Guardar".	El sistema actualiza el precio y el menú muestra el nuevo valor.
	2. Precio inválido	En caso que el nuevo precio sea 0, negativo o no sea un número. Cuando el encargado presione "Guardar".	El sistema no actualiza el precio y muestra "El precio debe ser un número mayor a 0".
	3. Pedidos ya registrados	En caso que existan pedidos registrados con el precio anterior. Cuando el encargado cambie el precio.	El sistema conserva el precio original en esos pedidos y aplica el nuevo precio solo a los pedidos nuevos.
CF-PEDI-0003	1. Franja válida	En caso que la franja elegida sea posterior a la hora actual. Cuando el estudiante confirme el pedido.	El sistema guarda la franja en el pedido y la muestra en el resumen.
	2. Franja ya pasada	En caso que la franja ya haya pasado o empiece en menos de 15 minutos. Cuando el estudiante vea las franjas.	El sistema la muestra deshabilitada y no permite elegirla.
	3. Sin franja	En caso que el estudiante no haya elegido una franja. Cuando presione "Confirmar pedido".	El sistema no registra el pedido y muestra "Selecciona una hora de recogida".
CF-PEDI-0004	1. Lista ordenada	En caso que existan pedidos en estado "Recibido". Cuando el encargado abra la lista de pendientes.	El sistema muestra número, nombre del estudiante, productos con cantidades y franja, ordenados por franja y luego por número.
	2. Sin pedidos	En caso que no haya pedidos en estado "Recibido". Cuando el encargado abra la lista de pendientes.	El sistema muestra "No hay pedidos pendientes".
	3. Pedido cancelado	En caso que un estudiante haya cancelado su pedido. Cuando el encargado abra la lista de pendientes.	El sistema no muestra ese pedido en la lista.
CF-PEDI-0005	1. Marcar como listo	En caso que el pedido esté en estado "Recibido". Cuando el encargado presione "Listo".	El sistema cambia el estado a "Listo", lo quita de pendientes y lo muestra en la lista "Para entregar".
	2. Pedido cancelado	En caso que el estudiante haya cancelado el pedido. Cuando el encargado intente marcarlo como listo.	El sistema no cambia el estado y muestra "Este pedido fue cancelado".
CF-PEDI-0006	1. Pedido existente	En caso que el número corresponda a un pedido del día. Cuando el estudiante lo consulte.	El sistema muestra el estado del pedido y la franja de recogida.
	2. Pedido listo	En caso que el pedido esté en estado "Listo". Cuando el estudiante lo consulte.	El sistema muestra "Tu pedido está listo, recógelo en la caja de pedidos anticipados".

Historia	Escenario	Contexto y evento	Resultado esperado
	3. Número inexistente	En caso que el número no corresponda a ningún pedido del día. Cuando el estudiante lo consulte.	El sistema muestra "No se encontró un pedido con ese número".
CF-PEDI-0007	1. Entrega registrada	En caso que el pedido esté en estado "Listo". Cuando el encargado elija el medio de pago y presione "Entregado".	El sistema cambia el estado a "Entregado" y guarda el medio de pago y la hora de entrega.
	2. Sin medio de pago	En caso que el encargado no haya elegido el medio de pago. Cuando presione "Entregado".	El sistema no cierra el pedido y muestra "Selecciona el medio de pago".
	3. Pedido no preparado	En caso que el pedido esté en estado "Recibido". Cuando el encargado intente registrar la entrega.	El sistema no lo permite y muestra "Primero marca el pedido como Listo".
CF-PEDI-0008	1. Cancelación permitida	En caso que el pedido esté en estado "Recibido". Cuando el estudiante presione "Cancelar" y confirme.	El sistema cambia el estado a "Cancelado" y lo quita de la lista de pendientes del encargado.
	2. Confirmación	En caso que el pedido esté en estado "Recibido". Cuando el estudiante presione "Cancelar".	El sistema pregunta "¿Seguro que deseas cancelar el pedido número X?" antes de cancelarlo.
	3. Pedido ya preparado	En caso que el pedido esté en estado "Listo" o "Entregado". Cuando el estudiante intente cancelarlo.	El sistema no lo permite y muestra "Tu pedido ya fue preparado y no se puede cancelar".
CF-MENU-0005	1. Registrar unidades	En caso que el encargado ingrese un número entero mayor o igual a 0. Cuando presione "Guardar".	El sistema guarda la cantidad y muestra las unidades restantes del producto.
	2. Descuento por pedido	En caso que el producto tenga unidades registradas. Cuando se confirme un pedido que lo incluya.	El sistema resta las unidades pedidas.
	3. Unidades agotadas	En caso que las unidades restantes lleguen a 0. Cuando se confirme el pedido que agota el producto.	El sistema marca el producto como "Agotado" automáticamente.
	4. Pedido cancelado	En caso que se cancele un pedido que incluía el producto. Cuando el sistema registre la cancelación.	El sistema suma de nuevo esas unidades y, si estaba agotado, lo vuelve a marcar como "Disponible".
CF-PEDI-0009	1. Definir cupo	En caso que el encargado ingrese un número entero mayor a 0. Cuando presione "Guardar".	El sistema aplica ese cupo a todas las franjas del día.
	2. Franja llena	En caso que una franja ya tenga el número máximo de pedidos. Cuando un estudiante vea las franjas disponibles.	El sistema muestra esa franja como "Llena" y no permite elegirla.
	3. Cupo liberado	En caso que se cancele un pedido de una franja llena. Cuando el sistema registre la cancelación.	El sistema vuelve a habilitar esa franja para nuevos pedidos.
CF-MENU-0006	1. Filtrar categoría	En caso que existan productos en la categoría elegida. Cuando el estudiante seleccione la categoría.	El sistema muestra solo los productos de esa categoría.
	2. Categoría vacía	En caso que no haya productos en la categoría elegida. Cuando el estudiante seleccione la categoría.	El sistema muestra "No hay productos en esta categoría hoy".
	3. Quitar filtro	En caso que haya un filtro aplicado. Cuando el estudiante seleccione "Todas".	El sistema muestra el menú completo.

ANEXO B. Registro de decisiones de arquitectura (ADR simplificado)

ADR	Contexto	Alternativas	Decisión y razón
ADR-01 Estilo de arquitectura	El proyecto exige 2 microservicios; el menú y los pedidos cambian por razones y usuarios distintos.	Monolito; 2 microservicios; 3 o más microservicios	2 microservicios (menu-ms y orders-ms): cumple el requisito, aísla fallos y mantiene la complejidad manejable para 3 personas.
ADR-02 Persistencia	Varias réplicas escriben al mismo tiempo; los datos deben sobrevivir a reinicios.	Cola en memoria; SQLite; PostgreSQL	PostgreSQL, una base por microservicio en un volumen: admite escrituras concurrentes de varias réplicas con bloqueos y transacciones (sección 3.2). SQLite solo sirve con una réplica.
ADR-03 Lenguaje y framework	El equipo conoce Python y debe explicar el código en la sustentación.	Python/FastAPI; Node.js/Express; Java/Spring Boot	Python con FastAPI: menos código, validación y documentación /docs automáticas.
ADR-04 Pago	La encuesta muestra que el pago no es una causa (93 % sabe que hay transferencia).	Pasarela en línea; pago al recoger	Pago al recoger: evita comisiones y trámites, y no ataca una causa real.
ADR-05 Identificación	Se requiere saber de quién es cada pedido sin datos personales innecesarios.	Login institucional; nombre y código	Nombre y código estudiantil, y el código se exige para cancelar: cumple la Ley 1581 con datos mínimos.
ADR-06 Acceso a las APIs	El navegador necesita llamar a dos servicios distintos.	Habilitar CORS en cada servicio; proxy nginx	Proxy nginx: una sola dirección para el usuario, sin CORS y con los microservicios ocultos tras la web.
ADR-07 Orquestación	La demanda se concentra en la hora pico; se necesita tolerar fallas y escalar.	Solo Docker Compose; Docker Swarm; Kubernetes	Docker Swarm: usa el mismo formato de Compose, se vio en la asignatura y es mucho más simple que Kubernetes para un clúster pequeño.

ADR	Contexto	Alternativas	Decisión y razón
ADR-08 Idioma del código	El docente pide trabajar en inglés; los usuarios son estudiantes de Ubaté.	Todo en español; todo en inglés; lo técnico en inglés y la interfaz en español	Lo técnico en inglés con Conventional Commits y la interfaz en español: sigue el estándar de la industria sin afectar a los usuarios.

ANEXO C. Evidencias de gestión

C.1 Tablero del proyecto. El backlog se gestiona en un tablero de Trello con listas Product Backlog, Sprint actual, En progreso, En pruebas y Hecho. Estado al 10 de noviembre de 2026:

Enlace del tablero: https://trello.com/b/M3eG364P

Insertar aquí la captura del tablero.

ID	Alias	Sprint	Puntos	Prioridad	Estado
CF-MENU-0001	Registrar producto	1	3	Alta	Sprint actual (11 nov)
CF-MENU-0002	Consultar menú del día	1	3	Alta	Sprint actual (11 nov)
CF-MENU-0003	Marcar producto agotado	1	2	Alta	Sprint actual (11 nov)
CF-PEDI-0001	Hacer pedido	2	5	Alta	Por hacer
CF-PEDI-0002	Número de pedido	2	2	Alta	Por hacer
CF-MENU-0004	Cambiar precio	2	2	Media	Por hacer
CF-PEDI-0003	Elegir hora de recogida	3	3	Alta	Por hacer
CF-PEDI-0004	Ver pedidos pendientes	3	3	Alta	Por hacer
CF-PEDI-0005	Marcar pedido listo	3	2	Alta	Por hacer
CF-PEDI-0006	Consultar estado del pedido	4	3	Media	Por hacer
CF-PEDI-0007	Registrar entrega y pago	4	2	Media	Por hacer
CF-PEDI-0008	Cancelar pedido	4	2	Media	Por hacer
CF-MENU-0005	Registrar unidades disponibles	5	3	Media	Por hacer
CF-PEDI-0009	Definir cupo por franja	5	3	Media	Por hacer
CF-MENU-0006	Filtrar menú por categoría	5	2	Baja	Por hacer
Total			40		

C.2 Matriz de riesgos. Probabilidad (P) e impacto (I): A alto, M medio, B bajo.

ID	Riesgo	P	I	Mitigación	Responsable
R1	A futuro, la cafetería no adopta el sistema o no tiene quién atienda la caja de pedidos	M	A	Fuera del alcance del prototipo; antes de lanzarlo, reunión con la encargada y piloto de una semana	Michell
R2	Pocos estudiantes usan el sistema	M	M	Código QR en mesas y difusión por WhatsApp; menú visible sin registrarse	Julian
R3	Pedidos que nadie recoge	M	M	Cancelación con código; registro por código; regla de no-show en versión 2	Michell
R4	Los pedidos saturan la preparación	M	A	Cupo por franja configurable (CF-PEDI-0009)	Michell
R5	Servidor o internet caído en la hora pico	B	A	La fila tradicional sigue; reinicio automático de contenedores	Julian
R6	Poca experiencia con Docker y FastAPI	M	M	Laboratorio de Docker; asistencia de IA; documentación	Julian
R7	Errores en el código generado con IA	M	A	Una prueba automática por criterio; revisión cruzada en cada pull request	Yenderson
R8	Los seis días de desarrollo no alcanzan	A	A	Sprints diarios; daily de 15 min; priorizar historias Alta (sprints 1 a 3) como producto mínimo	Julian
R9	Manejo indebido de datos personales	B	M	Solo nombre y código; aviso de privacidad (Ley 1581 de 2012)	Yenderson
R10	Diagnóstico basado en una muestra pequeña	A	M	Ampliar la encuesta a otros programas y al personal	Michell
R11	Datos inconsistentes o caídas por la complejidad de Swarm con varias réplicas	M	A	PostgreSQL con bloqueos; prueba de concurrencia con 3 réplicas; healthcheck y reinicio automático	Michell

C.3 Presupuesto detallado (COP de 2026). Supuestos del equipo: valor hora de referencia de $15.000; portátiles de $2.500.000 con vida útil de 5 años; tasa de cambio de $4.000 por dólar; plan de servidor de 1 vCPU y 2 GB a US$12 por mes [18].

Rubro	Cálculo	Tipo	Valor
Trabajo de desarrollo	(Michell 40 h + Julian 20 h + Yenderson 20 h) × $15.000	Especie	$1.200.000
Asesoría docente	Horas de clase y asesoría	Institucional	No se cuantifica
Uso de portátiles	3 × (2.500.000 / 60 meses) × 0,5 meses	Especie	$62.500

Rubro	Cálculo	Tipo	Valor
Internet	Parte del plan del hogar usada en el proyecto	Especie	$60.000
Software	Python, FastAPI, PostgreSQL, nginx, Docker, Docker Swarm, Git, Colab y Antigravity en planes gratuitos	Libre	$0
Equipo en la caja	Computador existente de la cafetería	Institucional	$0
Difusión	Códigos QR y afiches impresos	Efectivo	$40.000
Servidor, escenario A	Servidor o máquina virtual de la UdeC (requiere aprobación de TI)	Institucional	$0
Servidor, escenario B	US$12 × 12 meses × $4.000	Efectivo	$576.000
Imprevistos	10 % del efectivo	Efectivo	$4.000 (A) / $61.600 (B)
Total en especie			**$1.322.500**
Total en efectivo			**$44.000 (A) / $677.600 (B)**

C.4 Interacción de validación con usuarios. Encuesta “Servicio de cafetería en la hora del almuerzo – UdeC Ubaté”, aplicada en Google Forms el 6 de noviembre de 2026 a 14 estudiantes y difundida por WhatsApp; resultados en la Fig. 1 y en el diagnóstico [1]. El análisis usa las 14 respuestas recibidas hasta el 6 de noviembre, fecha de corte del diagnóstico. Después llegó una respuesta más (15 en total, como muestran las capturas); con ella los indicadores cambian menos de 7 puntos porcentuales y las conclusiones no se modifican.

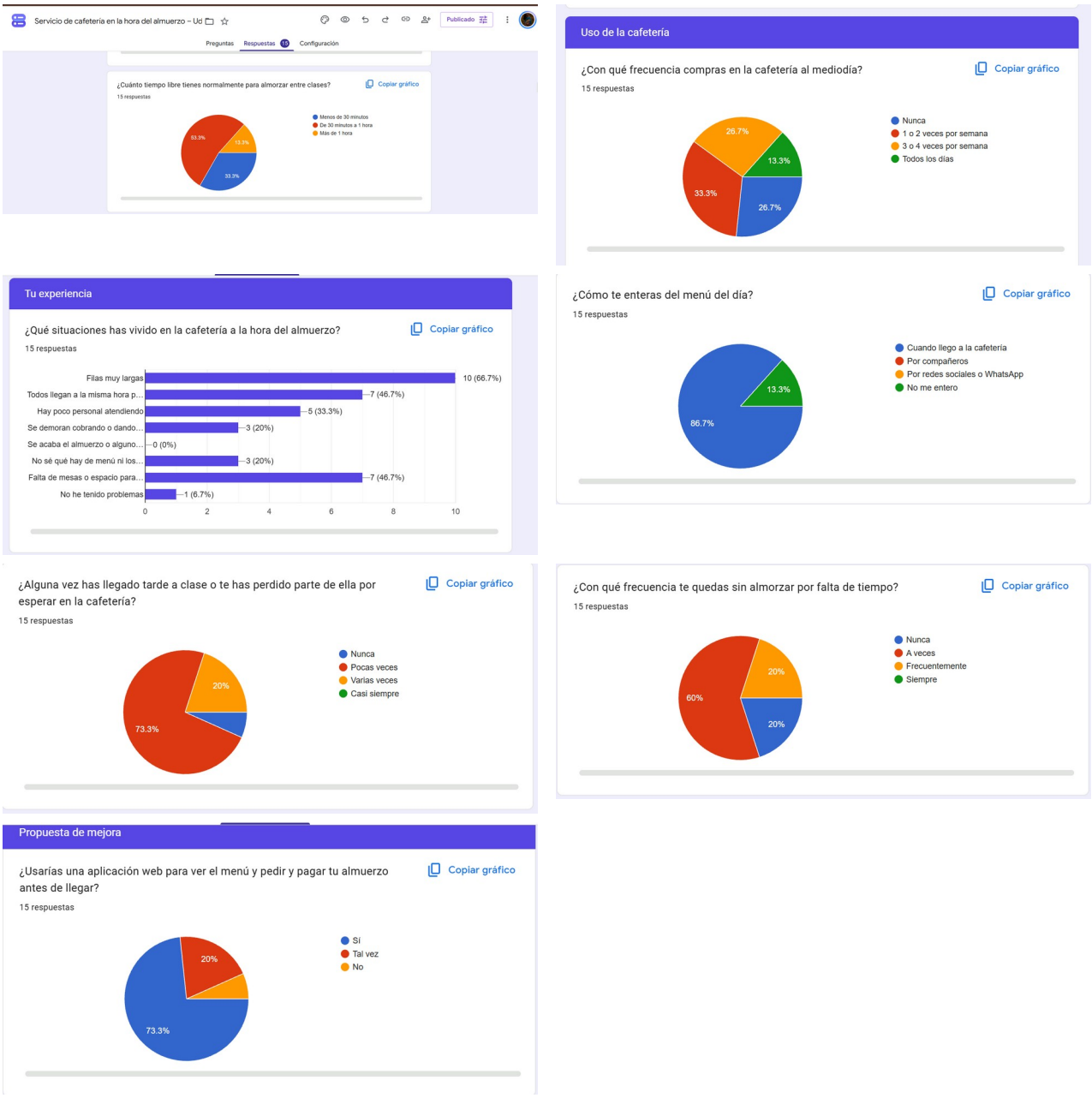


Fig. C1. Resumen de respuestas en Google Forms (capturas del 10 de noviembre de 2026).

Prueba de aceptación del prototipo (15 de noviembre de 2026): compañeros del grupo hacen pedidos desde el celular como estudiantes y un integrante atiende el panel como encargado. Registrar aquí participantes, tiempo para hacer un pedido (V6) y observaciones.

ANEXO D. Protocolo de pruebas del prototipo

Este protocolo se ejecuta durante los sprints, a medida que Antigravity genera cada parte y el equipo la revisa. La columna Obtenido se llena con el resultado real de cada prueba.

D.1 Prueba del algoritmo central (concurrencia). Registrar un producto con 10 unidades y una franja con cupo 5; con 3 réplicas de orders-ms, enviar 20 pedidos de 1 unidad al mismo tiempo (script con hilos). Esperado: 5 pedidos aceptados con números distintos y consecutivos, 15 rechazados por franja llena, 5 unidades restantes y ningún número repetido.

D.2 Tolerancia a fallas en Swarm. Con el sistema en uso, detener una réplica de orders-ms. Esperado: los pedidos siguen funcionando y docker service ps muestra que Swarm creó una réplica nueva.

D.3 Pruebas manuales:

ID	Acción	Esperado	Obtenido
PM-01	Abrir el menú sin productos	“La cafetería aún no ha publicado el menú de hoy”	
PM-02	Registrar “Almuerzo corriente”, almuerzo, 12000	Producto en estado Disponible	
PM-03	Registrar con precio 0	“El precio debe ser un número mayor a 0”	
PM-04	Registrar sin categoría	“Complete los campos obligatorios” y el campo	
PM-05	Registrar “ALMUERZO corriente” (duplicado)	“Ya existe un producto con ese nombre”	
PM-06	Marcar un producto como Agotado	Aparece “Agotado” y no se puede seleccionar	
PM-07	Filtrar por una categoría vacía	“No hay productos en esta categoría hoy”	
PM-08	Hacer un pedido válido	Número 001, estado Recibido y total correcto	
PM-09	Confirmar pedido sin productos	“Agrega al menos un producto”	
PM-10	Confirmar sin elegir franja	“Selecciona una hora de recogida”	
PM-11	Detener menu-db y hacer un pedido	“No fue posible verificar el menú, intenta de nuevo en unos minutos”	
PM-12	Marcar el pedido como Listo y consultarlo	“Tu pedido está listo, recógelo en la caja de pedidos anticipados”	
PM-13	Entregar un pedido en estado Recibido	“Primero marca el pedido como Listo”	
PM-14	Cancelar un pedido Listo	“Tu pedido ya fue preparado y no se puede cancelar”	
PM-15	Llenar una franja (cupo 2) y cancelar uno	La franja pasa a “Llena” y luego se habilita	

ANEXO E. Matriz de trazabilidad

Relaciona cada historia con la necesidad que atiende, los endpoints que la implementan, la pantalla donde se usa y la prueba que la verifica. Así se comprueba que todo requisito tiene código y prueba, y que todo el código responde a una necesidad real.

Historia	Necesidad	Microservicio y endpoints	Pantalla	Prueba	Estado
CF-MENU-0001	N1	menu-ms: POST /products	Encargado: Productos	test_menu.py	Sprint actual
CF-MENU-0002	N1	menu-ms: GET /menu	Estudiante: Menú de hoy	test_menu.py	Sprint actual
CF-MENU-0003	N4	menu-ms: PATCH /products/{id}/status	Encargado: Productos	test_menu.py	Sprint actual
CF-PEDI-0001	N2	orders-ms: POST /orders; POST /products/reserve	Estudiante: Tu pedido	test_orders.py	Por desarrollar
CF-PEDI-0002	N2	orders-ms: POST /orders	Estudiante: confirmación	test_orders.py (concurrencia)	Por desarrollar
CF-MENU-0004	N8	menu-ms: PATCH /products/{id}/price	Encargado: Productos	test_menu.py; precio conservado en test_orders.py	Por desarrollar
CF-PEDI-0003	N3	orders-ms: GET /slots; POST /orders	Estudiante: Hora de recogida	test_orders.py	Por desarrollar
CF-PEDI-0004	N5	orders-ms: GET /orders?status=RECEIVED	Encargado: Pendientes	test_orders.py	Por desarrollar
CF-PEDI-0005	N5	orders-ms: POST /orders/{number}/ready	Encargado: Pendientes	test_orders.py	Por desarrollar
CF-PEDI-0006	N3	orders-ms: GET /orders/{number}	Estudiante: Consultar pedido	test_orders.py	Por desarrollar
CF-PEDI-0007	N6	orders-ms: POST /orders/{number}/deliver	Encargado: Para entregar	test_orders.py	Por desarrollar
CF-PEDI-0008	N7	orders-ms: POST /orders/{number}/cancel; POST /products/release	Estudiante: Consultar pedido	test_orders.py	Por desarrollar
CF-MENU-0005	N4	menu-ms: PUT /products/{id}/stock; reserve; release	Encargado: Productos	test_menu.py; concurrencia (Anexo D.1)	Por desarrollar
CF-PEDI-0009	N5	orders-ms: GET y PUT /config; GET /slots	Encargado: Ajustes	test_orders.py	Por desarrollar
CF-MENU-0006	N1	menu-ms: GET /menu?category=	Estudiante: filtros	test_menu.py	Por desarrollar

Las necesidades N1 a N8 están en la Tabla A1. La columna Estado se actualiza al cerrar cada sprint.

ANEXO F. Glosario

Término	Significado en este proyecto
API REST	Forma en que los programas se piden datos por internet usando direcciones (URL) y métodos HTTP como GET y POST.
Microservicio	Programa pequeño e independiente que hace una sola parte del sistema (menu-ms o orders-ms) y tiene su propia base de datos.
Endpoint	Dirección de la API que hace una acción concreta, por ejemplo POST /orders.
JSON	Formato de texto con el que viajan los datos entre la página y los microservicios.
Contenedor	Paquete que incluye un programa con todo lo que necesita para correr igual en cualquier computador.
Dockerfile	Archivo con las instrucciones para construir la imagen de un contenedor.
Docker Compose	Herramienta que levanta varios contenedores juntos con un solo archivo; se usa para desarrollar.
Docker Swarm	Modo de Docker que une varias máquinas en un clúster y corre cada servicio en varias réplicas.
Réplica	Copia de un mismo servicio corriendo al tiempo; si una falla, las otras siguen atendiendo.
Franja	Intervalo de 15 minutos en el que el estudiante recoge su pedido; cada franja tiene un cupo máximo.
Cupo	Número máximo de pedidos que se aceptan en una franja, según lo que alcanza a preparar la cafetería.
Sección crítica	Parte del código que no pueden ejecutar dos procesos al mismo tiempo, como asignar el número de pedido.
Exclusión mutua	Garantía de que solo un proceso a la vez entra a la sección crítica; aquí se logra con bloqueos de PostgreSQL.
Historia de usuario	Descripción corta de una funcionalidad desde el punto de vista de quien la usa, con criterios de aceptación.
Sprint	Periodo corto de trabajo (un día en este proyecto) en el que se terminan algunas historias.
ADR	Registro de una decisión de arquitectura: el contexto, las alternativas y por qué se eligió una.